Information Security

Lecture 1: Foundations of Information Security

A textbook treatment

Based on the lectures, TPs and quiz of Imane Fouad

October 25, 2026

Contents

How to read this book **2**

1 Foundations of Information Security **3**

1.1 Precision, Properties, Risk 3
1.2 Services and Controls 5
1.3 Actors and Attack Classes 6
1.4 Malware by Mechanism 8
1.5 Domains, Access Control, GRC 9
Chapter takeaways 10

How to read this book

Each concept in this book is presented in the same rhythm, and every section closes with a *Go deeper* box (the exact details, numbers and edge cases a demanding examiner asks about) followed by *Hard questions, answered.*

1. **Motivation and intuition.** Why do we care? What problem does the idea solve, and what picture should you hold in your head?
2. **Formal statement.** The precise definition or theorem, set in a coloured box so it is easy to find when revising.
3. **Proof.** Every claim is proved. Longer proofs are split into labelled steps, and the one idea that makes the proof work is isolated in a *Key insight* box.
4. **Interpretation and discussion.** What the result really says, where it applies, and which misreadings to avoid.
5. **Examples.** From "trivially small" to "genuinely useful."
6. **Techniques, classic problems and gotchas.** Computational shortcuts, proof patterns, exam favourites with one-line solutions, and the mistakes everyone makes once.

Margin notes carry the informal, "say it out loud" version of what is happening in the main text. Every major section opens with a *What are we trying to get?* box that states the goal in one breath before any mathematics, and closes with a summary box.

Prerequisites. Comfort with binary and hexadecimal, elementary probability, and the modular arithmetic of integers; everything else (finite fields for AES, the number theory behind RSA, Shannon entropy) is built where it is needed.

Chapter 1

Foundations of Information Security

1.1 Precision, Properties, Risk

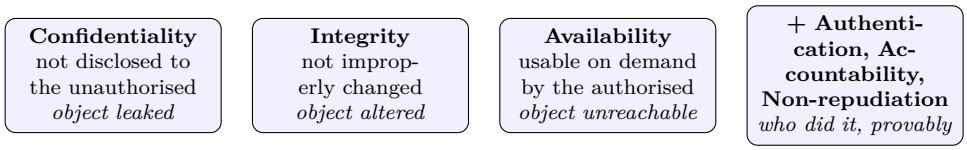
What are we trying to get?

Every later chapter is an instance of the vocabulary built here: an *asset* with a *property*, a *threat* exploiting a *vulnerability*, a *control* that preserves the property under a stated *threat model*. We want the definitions precise enough to classify any scenario, and the habit of naming all four parts of a security claim.

"Is this circle secure?" has no answer; "if I flood-fill outside the circle, does colour get inside?" does, and it depends on whether the boundary is closed. The two lessons: a security claim must name the adversary's capability before it can be judged, and the defence must be built systematically, because a freehand boundary has gaps. The course's three numbers each justify a later design choice: about 90% of attacks involve human error or behaviour (controls must bound the cost of a mistake), a breach costs $3.86M on average, and takes 191 days to discover (detection is not optional).

"Secure" is a relation: secure against what*, for* which asset*, with respect to* which property.

Definition 1.1.1 (Security properties (ISO 27001))**.** *Information security is the preservation of confidentiality, integrity and availability of information; authenticity, accountability, non-repudiation and reliability can also be involved.* **Confidentiality**: information is not made available or disclosed to unauthorised individuals, entities or processes. **Integrity**: data or resources are not improperly changed, deleted or corrupted; *data* integrity (records unaltered) versus *system* integrity (programs, configurations, processes intact). **Availability**: accessible and usable upon demand by an *authorised* entity. **Authentication**: a user or a data origin is accurately identifiable. **Accountability**: actions are traceable to those responsible. **Non-repudiation**: an actor cannot later deny an action to a third party; stronger than authentication, which only convinces the receiver.

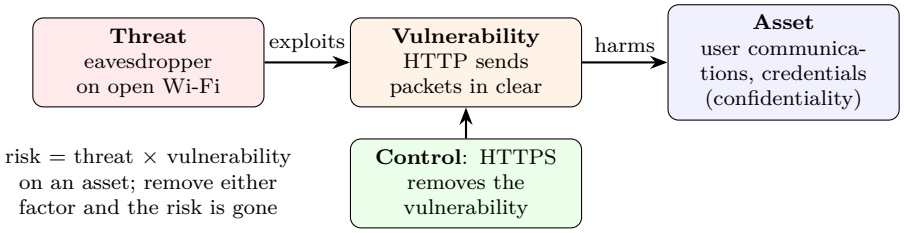


Reading the formula

To find the broken property, find the *object* the attacker touched and what the system assumed about it. Airport: Eve buys a ticket in Alice's name with a stolen card and forges a boarding pass in *her own* name; the person–pass check passes (Eve's ID, Eve's pass) and the pass–record check is skipped; a trusted document was altered: *integrity*, and no record of Eve flying exists. Targeted ads built on Alice's private browsing: *confidentiality*. The submission platform crashing before the deadline: *availability*, with no adversary at all. The properties are independent: each can fail alone, and Chapter 5 shows the cryptographic tools are equally independent (encryption gives C, a MAC gives I and authentication, a signature adds non-repudiation, nothing gives A).

Example 1.1.2 (Eve attacks Acme)**.** (1) Phishing yields a valid username and password: *confidentiality* of the credential, and *authentication* is undermined (Eve can now authenticate as the employee). (2) A buffer overflow injects code into a trusted server process that executes it as root: *system integrity*, and authentication of the actor. (3) Eve's code disables database logging: *accountability*. (4) She creates a database user by editing the table of authorised users: *integrity* of authorisation data and *authentication*. (5) Many machines flood the web server (*availability*) while she copies the database (*confidentiality*). Name properties, not techniques, and name all of them.

Definition 1.1.3 (Risk; threat model)**. Asset**: what we protect (data, systems, people). **Threat**: something that can cause damage. **Vulnerability**: an exploitable weakness. **Risk**: the potential that a threat exploits a vulnerability and harms an asset. Jim Anderson: security is "a well-informed sense of assurance that the information risks and controls are in balance". A **threat model** fixes (i) the asset, (ii) the vulnerability assumed and defended against, (iii) the attacker's capabilities and knowledge; only then can a defence be said to *prevent*, *detect*, *respond to* or *recover from* the attack. Risk treatment has four options: treat (mitigate), accept, transfer (insurance), avoid.



Key insight

Risk needs both factors: the same unpatched service is a risk on the Internet and none on an isolated machine. A control is judged *inside* its model: HTTPS answers interception on open Wi-Fi; it says nothing about phishing or a compromised laptop, which are other vulnerabilities with other attacker capabilities.

Go deeper: the details an examiner asks about

- Quiz 1 Q1–3 are one-property scenarios: reading a record without change (C), altering a result that stays accessible (I), a denial of service (A).
- "Upon demand by an authorised entity": a lockout that refuses legitimate users breaks availability without any attacker.
- Authentication has two senses in the course: user authentication (access control) and data-origin authentication (MACs, signatures); the ISO wording covers both.
- Adversaries are parameters of the model: a behavioural advertiser, Guccifer 2.0 (leaked DNC documents), and the NSA's own adversary, Snowden, the insider.

Hard questions, answered

Q. A stolen backup tape is encrypted. Which properties are affected?
A. Confidentiality preserved (strong cipher, safe key); availability of that copy lost, the organisation's availability depending on other copies; integrity of the original untouched.

Q. Why is non-repudiation not implied by authentication?
A. With a shared secret Bob is convinced Alice sent the message, but Bob could have forged it himself; a judge cannot tell. Non-repudiation needs something only Alice can produce: a signature.

Q. Threat model for a stolen password database?
A. Asset: passwords and accounts (confidentiality). Vulnerability: the verifiers are in the attacker's hands. Attacker: unlimited offline guessing with dictionaries. Control: per-user salt plus a slow hash (Chapter 5). Out of scope: observing passwords at login.

1.2 Services and Controls

What are we trying to get?

Separate what we want from how we get it, and classify every control on two axes, because the exam asks for both labels.

Definition 1.2.1 (Service versus control; two classifications)**.** A **service** (goal, property) is *what* we want: C, I, A, accountability, authentication. A **control** (mechanism) is *how*: encryption, firewalls, training; services are supported by controls. By implementation: **physical** (facility protection, guards, locks, monitoring, environmental controls, physical intrusion detection), **technical** (logical access control, cryptographic controls, security devices, user authentication, network intrusion detection, forensics), **administrative** (policies, standards, procedures and practice, personnel screening, awareness training). By function: **preventive** (prevents exploitation: encryption, a firewall rule, MFA), **detective** (warns of attempts: IDS, cameras, log review), **corrective** (repairs what was detected: restoring a known-good

image).

Scenario	Implementation	Function
MFA for company email	technical	preventive
Firewall blocks unauthorised connections	technical	preventive
Cameras record the building entrance	physical	detective
Phishing-awareness training	administrative	preventive
Backups restored after ransomware	technical	corrective
IDS alerts on suspicious traffic	technical	detective

Reading the formula

The implementation axis asks "what kind of thing is it"; the functional axis asks "when in the attack's life does it act" (before, during, after). The same word can live in two categories (a motion sensor is physical intrusion detection, a network sensor is technical); one control can serve two functions (a firewall that blocks *and* logs); and a control can be wrong for a service (encryption gives C, not A: ransomware is encryption aimed at availability). Defence in depth: controls of different categories so that one failure (a phished password) is caught by another (MFA, anomaly detection, least privilege).

Go deeper: the details an examiner asks about

- Backup is preparation; the *restore* is the corrective act; classify by the action named in the scenario.
- The three statistics map to the three functions: human error undermines prevention, 191 days is the absence of detection, \$3.86M is what correction limits.

Hard questions, answered

Q. Classify, both labels: a guard checking badges; a morning log review; lockout after five failures; a password-rotation policy; a disaster-recovery site.
A. Physical/preventive; administrative/detective; technical/preventive; administrative/preventive; technical (or physical)/corrective.

Q. One control for accountability and one for non-repudiation, with labels.
A. Protected audit logging: technical/detective. Digital signatures under a PKI: technical/preventive.

1.3 Actors and Attack Classes

What are we trying to get?

Who attacks, with what motive and resources, and what each attack class does to which property; precise enough to place a described incident in the right box.

A **hacker** is skilled; the *hat* names intent and legality: **black** (criminal, for gain or selling services on the dark web), **white** (authorised, finds vulnerabilities and helps fix them), **grey** (typically a former black hat who works with organisations but does not necessarily respect the law, e.g. tests without permission). By

motive and resources: **state-related** (very high resources; espionage or intrusion into another state's critical systems), **cybercriminals** (high resources; money), **cyber-terrorists** (destruction), **hacktivists** (ideology; defacements with a message), **script kiddies** (low capability; others' tools; amusement). A threat model needs both axes: script kiddies are stopped by patching known vulnerabilities; states buy unknown ones.

Attack class	Mechanism	Property hit
Social engineering	manipulate a person into revealing or acting; the vulnerability is the human	C, I (via the action)
DDoS	bots in a botnet flood an Internet-exposed target for hours to days; many legitimate-looking sources	A
Vulnerability exploit	weaknesses in OS, database, application (OWASP Top 10 lists web classes; Metasploit automates)	access or A
Unauthorised access	social engineering, brute force, *password cracking*: hashes already stolen, guesses tested offline	C, authentication
Privilege abuse	an insider with unnecessary privilege (no least privilege) and little logging (no accountability)	C, I, A
Ransomware	a payload: encrypt data, sell the key	A

Interpretation & discussion

Three entries carry design lessons. DDoS is hard to stop because it is *distributed*: blocking one source does nothing, so the control is capacity and upstream filtering. Password cracking is *offline*: login rate limits are irrelevant once the hash file is stolen; only the per-guess cost of the hash and the password's entropy matter. Privilege abuse has *two* enabling conditions and each is a failed principle: unnecessary privilege (least privilege) and missing logs (accountability).

Go deeper: the details an examiner asks about

- Grey hat is defined by *method*, not alternating intent; a white hat acting without authorisation is grey by this definition.
- A botnet is the product of a malware campaign and the instrument of an availability attack.

Hard questions, answered

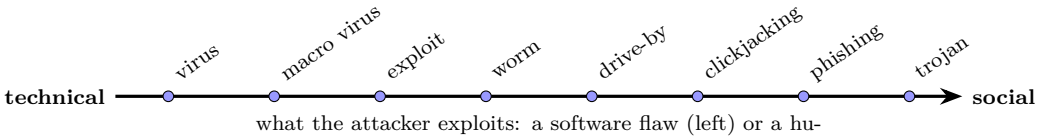
Q. Why does a six-character password survive online guessing with lockout but fall offline?
A. Online: a few tries per hour. Offline: $26^6 \approx 3 \times 10^8$ hashes computed in under a second against the stolen verifier; the only brake is the hash's per-guess cost.

Q. An admin reads customers' messages "to debug". Class and failed principles?
A. Privilege abuse; least privilege (debugging should not require content) and accountability (reads unlogged).

1.4 Malware by Mechanism

What are we trying to get?

Malware families as *mechanisms*, what each infects, how it spreads, who triggers execution, so that an incident can be classified and the right control is obvious.



what the attacker exploits: a software flaw (left) or a human decision (right); the control follows: patching and integrity monitoring vs training, procedures and interface design

Definition 1.4.1 (The families)**. Virus**: infects *programs*; modifies them to carry a copy of itself; executes when the host runs, with the host's permissions (virus = host + execution). Components: infection mechanism (the vector), trigger (a condition; a *logic bomb*), payload. **Macro virus**: document scripting; Melissa (1999) was a Word macro that accessed Outlook and mailed itself to contacts; macros now carry warnings and a separate `.xlsm` extension. **Worm**: seeks out *machines*; each infected host attacks others; spreads by network (web-application upload bugs, SSH dictionary attacks), shared media (USB autostart) or social means; strategies: *random* scanning (high traffic, noisy), *hit-list* (precompiled targets, stealthy; Stuxnet), *topological* (information on the victim), *local subnet* (behind NAT or firewall). Morris worm (1988): `fingerd` and `sendmail` bugs plus weak passwords; first conviction under the 1986 Computer Fraud and Abuse Act. **Rootkit**: with root privilege, modifies kernel code or data so that the OS lies (`ps` omits a process); *not* an infection method but stealth and continued access (a back door); needs root obtained by another attack; "runs at boot" alone is not a rootkit. **Drive-by download**: browser vulnerabilities install malware on a mere page visit (Angler kit, mid-2010s: visit, redirect, check for vulnerabilities, exploit, install). **Clickjacking**: input meant for one action performs another, by timing (a dialog appears under the "puppy" 200 ms later) or circumstance (a transparent authorisation page over a game); Chapter 2. **Phishing**: impersonate a trusted source to induce an action (click, open, reply, change a setting, transfer money); *spear* phishing targets specific victims with facts from reconnaissance; Google Docs 2017: a fake share invitation led to an OAuth consent screen, no malware, the goal was *authorisation*. **Trojan**: the victim runs it (`adorablecat.jpg.exe`, a fake ad, "install this plugin"); Zeus/Zbot banking trojan.

Key insight

Viruses infect executables to spread; worms infect machines to spread; trojans infect machines if you run them; rootkits infect kernels to avoid detection. Classical viruses are rare today because their one necessary act, modifying a binary, is cheaply prevented (no write access to program files) and detected (tracking binary hashes). Ransomware is a payload, not a vector.

Example 1.4.2 (Guess the type)**.** A: an outdated Windows service is exploited, the malware scans the LAN automatically, reaches 200,000 machines in 150 countries, encrypts files and demands Bitcoin: a *worm* with a *ransomware* payload. B: a compromised accounting-software update; spreads by a network exploit *and* stolen credentials; shows a ransom note but paying never recovers the files: *supply-chain* delivery, *worm* propagation, a *wiper* disguised as ransomware. Classify by mechanism, not by the note on the screen.

Technique: Phishing defences

Users: skepticism towards all communications; limit what you publish (the attacker's background material); inspect links; reach known services yourself. Organisations: training; chain-of-command procedures so that an email is never a valid channel for critical requests; email link screening, which shows all outgoing clicks, reveals trends and black-holes links at the forwarding stage (a detective control that becomes corrective).

Go deeper: the details an examiner asks about

- Rootkit detection must come from outside the lying kernel: boot clean media, compare against known-good images.
- Hit-list = targeted and stealthy; random = fast and loud; the slides pair the words deliberately.
- Consent-screen phishing defeats "inspect the link"; organisation-side controls matter.

Hard questions, answered

Q. Hashing binaries stops viruses but not worms. Why?
A. A virus must modify executables, so hashes change; a worm arrives as a new process through a network exploit and touches no existing program. Worm controls: patching, segmentation, credential hygiene.

Q. A program hides its files, survives reboot, and came as an email attachment the user opened. Vectors and categories?
A. Phishing delivery of a trojan; persistence is ordinary; hiding files by altering OS answers is a rootkit component, which needed root (an exploit, or the user running it as administrator).

1.5 Domains, Access Control, GRC

Technical domains: network security (traffic, firewalls), web security (applications, sessions, input), cryptography (encryption, hashing, signatures, PKI), access control (authentication, authorisation, least privilege), endpoint security (operating systems), cloud security (monitoring, detection, response). Governance domains: GRC and privacy. Chapters 2 and 3 are the privacy domain; Chapters 4 and 5 are cryptography.

Definition 1.5.1 (IAAA; GRC)**.** Access control decides who or what may access a resource and what they may do: *identification* (claim an iden-

tity), *authentication* (prove it), *authorisation* (what the identity may do), *accountability* (log and audit). Each fails separately: a badge reader identifies the badge, not the holder (add a PIN); a logged-in user reaching `/admin` is an authorisation failure; a shared account defeats accountability. **GRC**: governance (direction and oversight, policies, roles, strategy), risk (identify, assess likelihood and impact, treat/accept/transfer/avoid, monitor), compliance (laws, standards, contracts, evidence and audits). Web model: `subdomain.mysite.com/folder/page?id=5`; GET fetches pages and resources, POST submits forms and logins.

Go deeper: the details an examiner asks about

- Least privilege lives in authorisation; privilege abuse is its failure.
- References to cite: Whitman & Mattord; Stallings & Brown; Anderson's *Security Engineering*; ISO/IEC 27001 and 27002; NIST; OWASP.

Hard questions, answered

Q. Map a TLS connection to a bank onto IAAA.
A. Identification: the server name; authentication: the certificate chain (server) and password/MFA (user); authorisation: the account's permitted operations; accountability: transaction logs tied to the session.

Q. Cyber-insurance and refusing to store card numbers are which treatments?
A. Transfer; avoid.

Chapter takeaways

Summary: Foundations

- Asset, property, adversary, control: all four or it is not a claim. Properties are independent; find the tampered object.
- Risk = threat × vulnerability on an asset; model before defending; treat, accept, transfer, avoid.
- Service = what, control = how; label controls by implementation *and* function.
- Malware by mechanism: virus (programs), worm (machines), trojan (the user runs it), rootkit (the kernel lies); the social end of the axis exploits the person.
- IAAA for access control; GRC for governance.